

server.py

```
from flask import Flask, jsonify, request, abort
import storage

def create_app():
    app = Flask(__name__)
    @app.route('/media', methods=['GET'])
    def list_media():
        # optional category filter
        category = request.args.get('category')
        if category:
            items = storage.list_by_category(category)
        else:
            items = storage.list_all()
        return jsonify(items)
    @app.route('/media/search', methods=['GET'])
    def search_media():
        # exact name match
        name = request.args.get('name')
        if not name:
            return jsonify({'error': 'name parameter required'}), 400
        item = storage.search_by_name_exact(name)
        if not item:
            return jsonify({'error': 'not found'}), 404
        return jsonify(item)
    @app.route('/media/<item_id>', methods=['GET'])
    def get_media(item_id):
        item = storage.get_by_id(item_id)
        if not item:
            return jsonify({'error': 'not found'}), 404
        return jsonify(item)
    @app.route('/media', methods=['POST'])
    def create_media():
        if not request.is_json:
            return jsonify({'error': 'JSON required'}), 400
        payload = request.get_json()
        # basic validation
        name = payload.get('name')
        if not name:
            return jsonify({'error': 'name required'}), 400
        item = {
            'name': name,
            'author': payload.get('author', ''),
            'date': payload.get('date', ''),
            'category': payload.get('category', 'Uncategorized')
        }
        created = storage.create_item(item)
        return jsonify(created), 201
    @app.route('/media/<item_id>', methods=['DELETE'])
    def delete_media(item_id):
        removed = storage.delete_item(item_id)
        if not removed:
            return jsonify({'error': 'not found'}), 404
        return jsonify({'deleted': True})
    @app.route('/media/<item_id>', methods=['PUT'])
    def update_media(item_id):
        if not request.is_json:
            return jsonify({'error': 'JSON required'}), 400
        payload = request.get_json()
        # Accept name, author, date, category
        updates = {
            'name': payload.get('name'),
            'author': payload.get('author'),
            'date': payload.get('date'),
            'category': payload.get('category')
        }
        # remove None keys
        updates = {k: v for k, v in updates.items() if v is not None}
        updated = storage.update_item(item_id, updates)
        if not updated:
            return jsonify({'error': 'not found'}), 404
```

```
        return jsonify(updated)
    return app
if __name__ == '__main__':
    app = create_app()
    app.run(host='127.0.0.1', port=5000)
```